

Git Revert Reset

GIT Reset

`git-reset` - Reset current HEAD to the specified state

Soft Reset (Default usage)

Moves HEAD: Points HEAD to the specific commit.

Does not modify the working directory: Files remain unchanged.

Preserves the staging area: Changes in the index (staged files) remain staged.

Use case: When you want to undo a commit but keep your changes staged so you can make adjustments or recommit.

```
git reset --soft <commit-hash>
```

Mixed Reset - Git uses mixed by default

Moves HEAD: Points HEAD to the specific commit.

Does not modify the working directory: Your working directory files remain unchanged.

Clears the staging area: All changes that were staged (in index) become unstaged but still remain in the working directory.

Hard Reset

Moves HEAD: Points HEAD to the specified commit.

Resets the working directory: All changes in the working directory are discarded and replaced with the content of the target commit.

Clears the staging area: The index is reset to match the target commit.

Use case: When you want to completely discard changes in the working directory and start fresh from a previous state.

```
git reset --hard <commit-hash>
```

Final

`git reset --soft` → Moves HEAD, keeps changes staged.

`git reset --mixed` → Moves HEAD, keeps changes in working directory, unstages them.

`git reset --hard` → Moves HEAD, discards all changes (staged + working directory).

Git Revert

Purpose: To undo a specific commit without altering the commit history.

Creates a new commit: Revert creates a new commit that negates the changes introduced by the target commit.

Safe for shared branches: Since revert preserves the commit history, it is ideal for undoing changes in public or shared branches.

```
git revert <commit>
```

```
git revert <commit1> <commit2> <commit3>
```

```
git revert abc123
```

Rebase

In Git, rebase is a command used to reapply commits from one branch onto another, effectively rewriting the commit history. Instead of merging changes (which creates a merge commit), rebasing replays the commits from the source branch on top of the target branch.

```
git rebase <target-branch>
```

Note - always perform on new branch - never do on main/master branch

When to Use

- Clean Linear History

- Update a Feature Branch

When NOT to Use

- On Shared/Public Branches

- When You Need Merge Traceability

Git Merge

In Git, merge is a command used to integrate changes from one branch into another. It combines the commit histories of the branches, creating a merge commit if necessary to record the integration.

Unlike rebase, merging preserves the history of the original branches.

```
git merge <source-branch>
```

Steps

git checkout main

git merge feature

If any conflicts

git status

git add <file>

git commit

git log --oneline

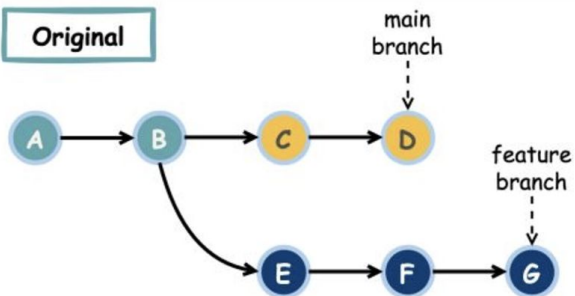
git push

When to use

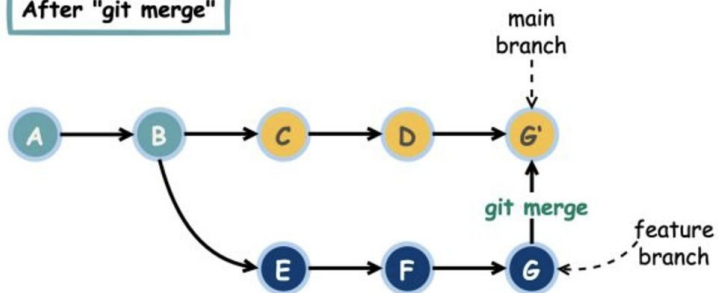
Preserve Branch History

Team Collaboration

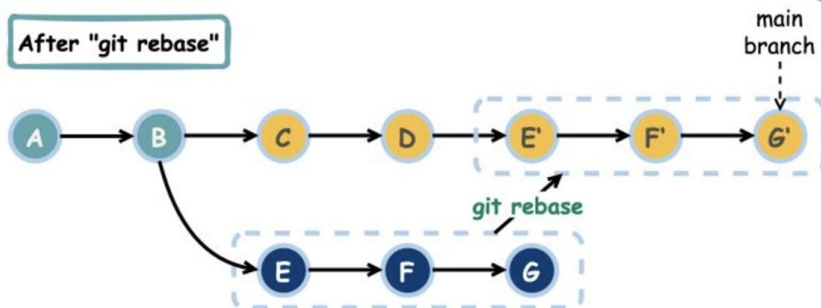
Original



After "git merge"



After "git rebase"



Final

When to do rebase and when to do merge

If new branch is created and now main branch got some new update then do rebase

```
git checkout featureA
```

```
git pull --rebase origin main
```

When new feature is ready then merge to main branch

```
git checkout main
```

```
git pull origin main
```

```
git merge featureA
```

```
git push origin main
```

Cherry pick

Cherry-pick

First

`git checkout main`

`git pull origin main`

`git cherry-pick <commit-hash-of-F>`

Stash

`git stash`

`git stash list`

`git stash pop`

Use Case

If you are on new branch and main branch got some new commits

```
git pull origin main (fetch + merge)
```

or

```
git pull --rebase origin main
```